

EXODUS OPS PLATFORM (EOP)

Sprint Plan — v0.1 Core Communication Layer

Plan de Implementación y Estabilización

Exodus Systems Inc. — Engineering Division

Documento: SPRINT-EOP-v01-001 (Actualizado)

Milestone: M1 — Core Communication Layer

Squad: 3 Ingenieros | Duración restante: ~2 Semanas

1. Análisis del Scope — EOP v0.1

Este documento presenta el plan de ejecución para la versión v0.1 del Exodus Ops Platform. La fase de diseño (Semana 1) está completada: todos los ADRs (001–006) están aprobados, el repositorio tiene CI operativo, y los diagramas y templates están en el repo. Este documento cubre exclusivamente el trabajo pendiente de implementación y estabilización.

1.1 Objetivo de Release

Un Vault Operator puede registrar nodos bunker, enviar comandos a través de la red y recibir respuestas con observabilidad básica del sistema, todo sin intervención manual en la capa de comunicación.

1.2 Epics y User Stories en Scope

Epic	Descripción	User Stories	Prioridad
E1	C++ TCP Server — Core del servidor de comunicaciones	US-101, US-102, US-103, US-104	CRITICAL PATH
E2	C Client Library — Abstracción para consumidores del protocolo	US-105, US-106, US-107	CRITICAL PATH
E3	Containerization & Kubernetes — Despliegue reproducible	US-108, US-109	HIGH
E4	Basic Observability — Trazas distribuidas en SigNoz	US-110	HIGH

1.3 NFRs Aplicables desde v0.1

NFR	Requerimiento	Umbral	Verificación
NFR-1	Latencia P99 del servidor C++	< 200 ms	OTel trace duration
NFR-1	Cientes TCP concurrentes mínimos	≥ 10 sin degradación	Load test
NFR-2	Graceful shutdown en SIGTERM	100% de los casos	Exit code 0, sin errores
NFR-3	RAII para todos los recursos	BLOCKING	Code review + ASan
NFR-3	Zero memory leaks	BLOCKING	ASan --leak-check en CI
NFR-3	Zero data races	BLOCKING	TSan en CI
NFR-3	Test coverage ≥ 90%	Required	gcov / llvm-cov en CI
NFR-3	Clang-Tidy: zero critical warnings	Required	clang-tidy en CI
NFR-3	Docker image runtime ≤ 50 MB	Required	docker images en CI
NFR-3	K8s manifests pasan validación	Required	kubeval/kubeconform en CI
NFR-4	Logs structured JSON a stdout	Obligatorio	Formato en runtime
NFR-4	Cada operación emite OTel span	Obligatorio	Visible en SigNoz < 30s

2. Notas Técnicas para Implementación

2.1 Estrategia de Sanitizers

ASan + TSan son gates obligatorios en CI para cada PR. No se puede mergear código con findings. Valgrind se usa como verificación pre-release durante la fase de estabilización (Semana 3). Ambas herramientas son complementarias: ASan/TSan corren con ~2x slowdown; Valgrind provee análisis más profundo con 20-50x slowdown.

3. Decisiones Arquitectónicas (Resumen)

Todas las decisiones están documentadas en /docs/adr/. Resumen para referencia rápida durante la implementación:

ADR	Decisión	Implicancia para Implementación
ADR-001	BSD/POSIX sockets directos	Implementar framing y partial reads manualmente. RAII obligatorio para fds.
ADR-002	Thread Pool (16 workers default)	Work queue con mutex + cond_var. NodeRegistry con shared_mutex. Pool size via EOP_THREAD_POOL_SIZE.
ADR-003	Length-prefix 4B BE + JSON	Envelope 10 bytes fijo. Types: REGISTER(0x01), ACK(0x02), ERROR(0x03), QUERY_NODE(0x04), LIST_NODES(0x05), HEARTBEAT(0x06). nlohmann/json para C++, cJSON para C.
ADR-004	Single service: eop-server	Monolito modular. SocketAcceptor → ThreadPool → MessageHandler → Handlers → NodeRegistry.
ADR-005	OTel + SigNoz desde v0.1	opentelemetry-cpp SDK. Span por operación. Logs JSON con trace_id. OTel Collector como container.
ADR-006	Docker multi-stage + K8s	Runtime image ≤ 50 MB. ENTRYPOINT JSON array. Graceful shutdown SIGTERM. Liveness + readiness probes.

4. Grafo de Dependencias entre User Stories

US	Nombre	Depende de	Bloquea a
US-101	Node connection & welcome handshake	ADR-002, ADR-003	US-102,103,104,105+
US-102	Concurrent multi-client support	US-101	US-104 (parcial)
US-103	Node registration	US-101	US-104, US-110
US-104	Disconnection detection & state update	US-101, US-103	US-110
US-105	Connection abstraction (client lib)	ADR-003 (protocolo)	US-106, US-107
US-106	Command send/receive	US-105, US-101	US-107
US-107	Clean disconnection & resource release	US-105	(ninguno)
US-108	Reproducible deployment (Docker/K8s)	US-101 (compilable)	US-109
US-109	Automatic pod recovery	US-108	(ninguno)
US-110	Distributed traces in SigNoz	US-101, US-103, US-108	(ninguno)

Camino crítico: US-101 → US-103 → US-104 → US-110 → Integración E2E. Cualquier retraso en US-101 impacta directamente en todo el sprint.

5. Plan de Sprint — Trabajo Pendiente

La fase de diseño (30%) está completada. El trabajo restante cubre implementación (50%) y estabilización (20%). Se asignan los 3 ingenieros del squad como Eng-A, Eng-B y Eng-C.

Semana 1 — Tareas Pendientes

Las siguientes tareas de la fase de diseño aún no se completaron:

Tareas pendientes de Semana 1

Asignado	Tarea	Entregable	Size
Eng-C	Dockerfile multi-stage skeleton (build + runtime). docker-compose.yml con server placeholder + OTel Collector + SigNoz. Verificar que el stack levanta.	Docker base	S
Eng-C	Definir estructura de logs JSON. Configurar OTel Collector (otel-collector-config.yaml). Definir span names y atributos para cada operación. Draft del runbook para SigNoz.	Config OTel	S
Eng-A	Redactar issues de E1: US-101, US-102, US-103, US-104 con ACs completos y campos obligatorios del SW Agreement.	Issues E1	S
Eng-B	Redactar issues de E2: US-105, US-106, US-107 con ACs completos y campos obligatorios del SW Agreement (Workflow Status, Team, Milestone, Priority, Size, Estimated Days, Target Date).	Issues E2	S
Eng-C	Redactar issues de E3 y E4: US-108, US-109, US-110 con ACs completos y campos obligatorios.	Issues E3/E4	S
TODOS	Design Review Session: revisar los 10 issues redactados con ACs. Validar que los acceptance criteria de cada US son alcanzables con el diseño aprobado.	Design aprobado	S

⚠ PREREQUISITO OBLIGATORIO: Los 10 issues deben estar redactados y revisados en GitHub antes de iniciar la implementación. Ningún trabajo de código comienza sin issues con ACs verificables.

SEMANA 2 — Fase de Implementación (50%)

Objetivo: E1 (TCP Server) y E2 (Client Library) implementados con tests unitarios. Containerización funcional. Inicio de observabilidad.

Días 6–7 — Server Core + Client Library Base

Asignado	Tarea	Entregable	Size
Eng-A	US-101: Socket server core. bind/listen/accept. Welcome handshake con version, timestamp, session_id. Puerto configurable via EOP_PORT. Idle timeout configurable. Logging de nuevas conexiones. Unit tests + integration test (100 iteraciones connect/handshake/disconnect).	US-101 Done	M
Eng-B	US-105: eop_connect(host, port) con handle opaco eop_client_t*. eop_last_error(). Connection timeout configurable. Header C99-compatible. Compilación como .so y .a. Unit test de conexión fallida con ASan.	US-105 Done	M
Eng-C	US-108 (parcial): Dockerfile multi-stage completo. ENTRYPOINT JSON array. Graceful shutdown handler (SIGTERM → stop accept → wait 15s → exit 0). docker-compose con server + OTel Collector + SigNoz. Makefile target deploy-local.	Docker funcional	M

Días 8–9 — Concurrencia + Registro + Comandos

Asignado	Tarea	Entregable	Size
Eng-A	US-102: Modelo de concurrencia (thread pool). Work queue thread-safe. EOP_MAX_CLIENTS configurable. Tests: 100 clientes concurrentes sin timeout, verificación de fd leaks con /proc/PID/fd, TSan zero races. Inicio de load test 1000 clientes.	US-102 Done	L
Eng-B	US-103: Implementar mensajes REGISTER, ACK, ERROR_DUPLICATE, QUERY_NODE, LIST_NODES según ADR-003. Registry in-memory. Unit tests: registro exitoso, duplicado, query existente/no existente, list multiple nodes. US-106: eop_send_command() + eop_response_t + eop_response_free().	US-103 + US-106	L
Eng-C	US-110 (inicio): Integrar opentelemetry-cpp SDK en el servidor. Crear spans para REGISTER, QUERY_NODE, LIST_NODES, connection, disconnection. Atributos: operation.name, node.id, duration_ms, result, session.id. Logs structured JSON con trace_id.	OTel base	L

Día 10 — Heartbeat + Disconnection + Client Cleanup

Asignado	Tarea	Entregable	Size
Eng-A	US-104: Heartbeat mechanism (EOP_HEARTBEAT_INTERVAL configurable, default 5s). Detección de desconexión < 10s. Transición a OFFLINE. Logging del evento. Reconexión sin restart. Integration test del ciclo completo.	US-104 Done	M
Eng-B	US-107: eop_disconnect() con shutdown(SHUT_RDWR) + close(). Safe no-op con NULL. ASan test 1000 ciclos. TSan test 10 threads x 100 ciclos. Documentar undefined behavior post-disconnect en header. Crear examples/connect_basic.c.	US-107 Done	M
Eng-C	US-108 (K8s): Deployment + Service manifests. ConfigMap para toda la configuración. Verificar kubectrl apply -f k8s/ sin errores. CI step para build de imagen en cada PR. kubeconform validation en CI.	US-108 Done	M

Gate de Semana 2: US-101 a US-107 implementados con tests unitarios. Docker build exitoso. CI green con ASan/TSan. K8s manifests válidos.

SEMANA 3 — Implementación Final + Estabilización (20%)

Objetivo: US-108 a US-110 completos. Integración end-to-end verificada. Documentación actualizada. Release tag en git. Todos los acceptance criteria verificables.

Días 11–12 — Observabilidad + Pod Recovery + Load Tests

Asignado	Tarea	Entregable	Size
Eng-A	US-102 AC8: Load test completo (1000 clientes, 10000 requests cada uno). Verificar zero server errors y zero client timeouts. Documentar resultados. Fix de cualquier race condition detectada por TSan bajo carga.	Load test pass	M
Eng-B	US-109: Configurar liveness probe (TCP check, 10s, failureThreshold:3) y readiness probe. Verificar restart < 10s tras crash. Integration test: delete pod → restart → reconnect → register. Verificar client recibe error limpio en crash.	US-109 Done	M
Eng-C	US-110: Completar integración SigNoz. Verificar spans visibles en < 30s. Integration test: REGISTER genera exactamente un span con operation.name y result correctos. Correlación trace_id en logs. Runbook en README.	US-110 Done	M

Días 13–14 — Integración E2E + Sanitizers + Documentación

Asignado	Tarea	Entregable	Size
Eng-A	Corrida completa de Valgrind --leak-check=full sobre test suite. Fix de cualquier leak. Verificar cppcheck / scan-build zero warnings. Corrida final de TSan con carga concurrente máxima. Verificar NFR-1: P99 < 200ms.	Sanitizers clean	M
Eng-B	Integration tests E2E: C client → connect → register → query → list → heartbeat → disconnect. Smoke test automatizado (make deploy-local + test). Verificar cobertura ≥90%. Escribir tests faltantes para alcanzar umbral.	E2E tests pass	M
Eng-C	Documentación Doxygen completa en todos los .hpp. README del servidor: build, run, test suite, deploy. README de la client library: build, link, usage. Sección de runbook: navegar de node_id en SigNoz a logs correspondientes.	Docs completos	M

Día 15 — Release Day

Asignado	Tarea	Entregable	Size
Eng-A	CI final green en release branch. Verificar docker image ≤ 50MB. Crear release tag v0.1.0 con Semantic Versioning.	Tag v0.1.0	S
Eng-B	CHANGELOG.md completo con todos los cambios de v0.1. Verificar que todos los issues en GitHub tienen Workflow Status = Done con todos los campos obligatorios.	CHANGELOG	S
Eng-C	Smoke test final: make deploy-local levanta todo el stack. Verificar conexión, handshake, registro, query, trazas en SigNoz. Validar que el stack es reproducible en ambiente limpio.	Smoke test pass	S
TODOS	Revisión final del squad: cada US tiene al menos un peer review aprobado, CI green, documentación actualizada, y todos los acceptance criteria son verificables.	v0.1 RELEASE	S

Gate de Release v0.1: US-101 a US-110 Done. CI green. Zero leaks (ASan). Zero races (TSan). Cobertura ≥90%. Docker image ≤50MB. K8s manifests válidos. EOP v0.1 desplegable. CHANGELOG actualizado. Tag v0.1.0 en git.

6. Resumen de Asignación por Ingeniero

Semana	Eng-A (Server Core)	Eng-B (Protocol + Client)	Eng-C (Infra + OTel)
S1 pend.	Issues E1	Issues E2	Docker, docker-compose, config OTel, issues E3/E4
S2	US-101, US-102, US-104 (server: socket, concurrencia, heartbeat)	US-105, US-103, US-106, US-107 (client lib + registro + commands)	US-108, US-110 inicio (Docker/K8s completo + OTel SDK)
S3	Load tests, Valgrind, sanitizers, release tag	US-109, integration tests E2E, cobertura, CHANGELOG	US-110 completo, documentación, runbook, smoke test

6.1 Distribución de Carga

La distribución de carga estimada para el trabajo restante es: Eng-A ~37%, Eng-B ~30%, Eng-C ~33%. Eng-A lleva el camino crítico (server core). Eng-C tiene mayor carga en tareas pendientes de Semana 1 (Docker + OTel config + issues E3/E4). Si se identifica un desbalance durante la ejecución, las tareas de estabilización de Semana 3 se redistribuyen.

7. Riesgos Identificados y Mitigación

#	Riesgo	Probabilidad	Mitigación
R1	Setup de opentelemetry-cpp SDK toma más de lo esperado (dependencias, build con CMake)	Alta	Eng-C empieza a investigar temprano. Si el SDK nativo no compila en Día 7, fallback a exportar métricas via stdout JSON y post-procesar con OTel Collector.
R2	Load test de 1000 clientes falla por límites del SO (ulimit, file descriptors)	Media	Configurar ulimit -n 65535 en Dockerfile y en CI runner. Documentar pre-requisitos de SO en README.
R3	Graceful shutdown (SIGTERM) no se propaga correctamente en Docker	Baja	Usar ENTRYPOINT JSON form. Verificar PID 1. Si persiste, usar tini como init process.
R4	Valgrind reporta leaks en bibliotecas de terceros (OTel SDK, system libs)	Media	Usar suppression file de Valgrind para leaks conocidos de terceros. Documentar en el repo.

8. Checklist de Compliance con SW Agreement

Requerimiento SW Agreement	Cubierto en	Estado
GPG-signed commits (Sec 3.1)	Configurado	✓ Activo
Conventional Commits (Sec 3.1)	CI	✓ Activo
Branch naming (Sec 3.2)	Policy	✓ Activo
clang-format strict (Sec 4.1)	CI	✓ Activo
clang-tidy zero warnings (Sec 4.1)	CI	✓ Activo
cppcheck / scan-build (Sec 4.1)	S3	✓ Planificado
ASan / TSan (Sec 4.1)	CI	✓ Activo
SonarQube zero findings (Sec 4.1)	CI	✓ Planificado

Naming conventions C/C++ (Sec 4.2)	Code review	✓ Activo
Doxygen documentation .hpp (Sec 4.4)	S3	✓ Planificado
Test coverage ≥90% (Sec 5)	S3	✓ Planificado
Campos obligatorios en issues (Sec 6.2)	S1 pendiente	⚠ Pendiente
Updates en issues In Progress (Sec 6.5)	Desde S2	✓ Planificado
CHANGELOG (Sec 3.4)	Día 15	✓ Planificado
No merge directo a main (Sec 3.4)	Branch policy	✓ Activo
Peer review requerido (Sec 6.6)	Cada PR	✓ Activo
Semantic Versioning (Sec 3.4)	Día 15: v0.1.0	✓ Planificado